

Essentia Danceability Plugin

Oriol Freixa

June 18, 2026

1 Overview

We followed the Essentia plugins setup, built the required JUCE/Essentia dependencies, and created a new Projucer plugin from the RMS template. We renamed the project to **Danceability** and adapted the RMS analyzer into a VST3 analyzer based on Essentia's **Danceability** algorithm.

The source code is available in the GitHub repository [OriolFreixa/essentia_plugin_actsm](https://github.com/OriolFreixa/essentia_plugin_actsm).

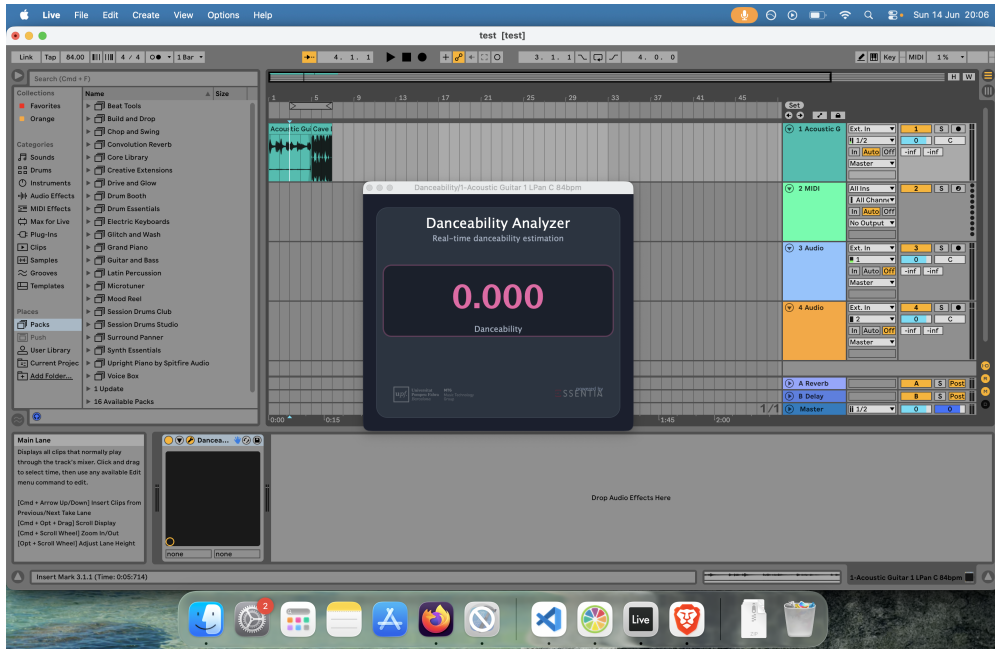


Figure 1: Danceability plugin loaded in Ableton Live.

2 Implementation

We replaced the RMS algorithm with **Danceability**. Since this descriptor needs a longer signal than a normal DAW audio callback, we adapted the processor so that it stored a rolling mono analysis window and computed the descriptor periodically instead of per block.

```
danceability = AlgorithmFactory::create("Danceability");
danceability->input("signal").set(essentiaBuffer);
danceability->output("danceability").set(danceabilityValue);
danceability->output("dfa").set(dfaValues);
```

We connected both Essentia outputs: the scalar **danceability** value is shown in the UI, while the **dfa** vector is kept internally because it is part of the algorithm output but not useful as a main user-facing value.

```

essentiaBuffer.resize(previousSize + numSamples);
// Mix host input to mono and append it to the rolling window.

if (essentiaBuffer.size() > analysisWindowSamples)
    essentiaBuffer.erase(essentiaBuffer.begin(),
        essentiaBuffer.begin() + samplesToDrop);

if (essentiaBuffer.size() >= analysisWindowSamples)
    danceability->compute();

```

3 Parameters

We exposed `minTau`, `maxTau`, and `tauMultiplier` because they are real Essentia Danceability parameters and make the analysis tunable from the host. We used `AudioProcessorValueTreeState` so the values can be automated and saved with the session.

```

params.push_back(std::make_unique<juce::AudioParameterFloat>(
    juce::ParameterID {"minTau", 1}, "minTau",
    juce::NormalisableRange<float> {50.f, 1000.f, 10.f}, 310.f));

params.push_back(std::make_unique<juce::AudioParameterFloat>(
    juce::ParameterID {"maxTau", 1}, "maxTau",
    juce::NormalisableRange<float> {1000.f, 8800.f, 100.f}, 8800.f));

params.push_back(std::make_unique<juce::AudioParameterFloat>(
    juce::ParameterID {"tauMultiplier", 1}, "tauMultiplier",
    juce::NormalisableRange<float> {1.01f, 1.5f, 0.01f}, 1.1f));

```

We also added `Analysis Window` as a useful extra parameter. It is not an Essentia Danceability parameter, but it controls how many seconds of recent audio we keep before analysis, which makes the plugin more practical in a DAW.

```

params.push_back(std::make_unique<juce::AudioParameterFloat>(
    juce::ParameterID {"analysisWindowSeconds", 1}, "Analysis Window",
    juce::NormalisableRange<float> {1.f, 15.f, 0.1f}, 10.f));

const auto analysisWindowSeconds =
    apvts.getRawParameterValue("analysisWindowSeconds")->load();

analysisWindowSamples =
    static_cast<size_t>(currentSampleRate * analysisWindowSeconds);

```

4 Interface And Test

We replaced the RMS labels with a compact Danceability Analyzer interface, added sliders for the exposed parameters, and kept the UPF and Essentia logos. We tested the plugin in Ableton Live using the included test project and recorded a short demo video. The video is available in `imgs_and_vids/danceability-demo.mov`.

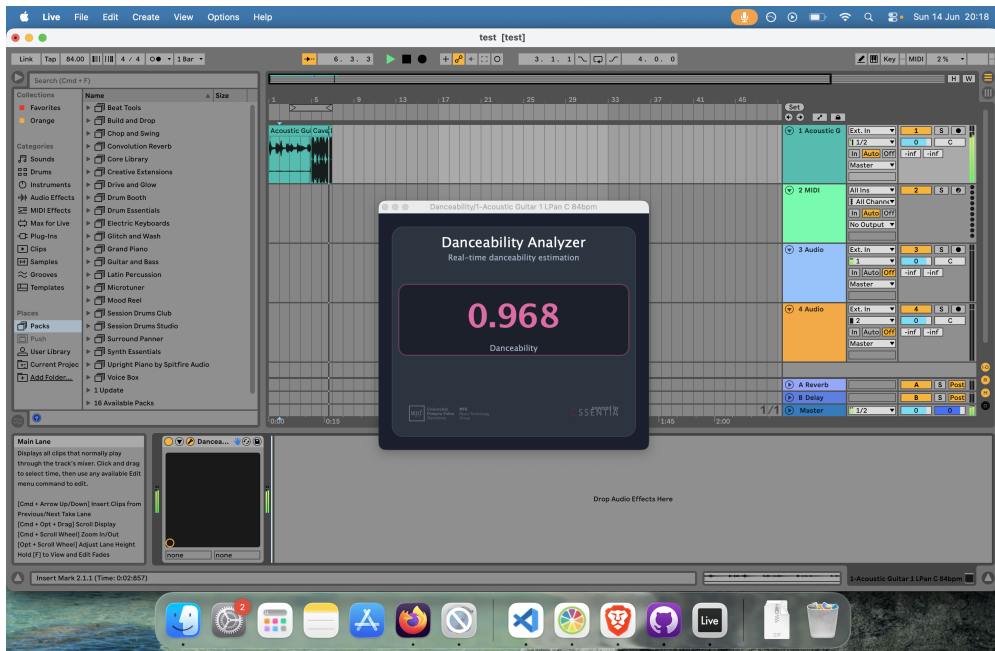


Figure 2: Analyzer running and showing a non-zero Danceability value.